

AI Open Source Mentor++

Project Design & Capability Document

Repository `manideep1799/MUTAV2` · branch `claude/git-quest-clone-z0qti3`

A code-intelligence system that indexes a GitHub repository into two parallel representations — a **deterministic dependency graph** (tree-sitter to NetworkX) and a **semantic vector index** (local BGE embeddings to ChromaDB) — then serves onboarding, change-impact, and engineering-management features on top of both. This document records what is built, how each piece works, which parts are verified against a live system, and which parts are not.

Scope and honesty of this document

Every claim here was checked against the source on this branch at the time of writing. Where the running code differs from the project's own design documents, this document follows the **code** and flags the divergence explicitly. Section 8 documents a significant one: the Mutagent optimization loop that the design narrative is built around does not exist in this repository.

Property	Value
Backend	Python 3.11 · FastAPI · Uvicorn
Frontend	React 19 · TanStack Start (SSR) · Vite · Tailwind
Generation model	llama-3.1-8b-instant via Groq (optionally llama3.1:8b via Ollama for RAG answers)
Embeddings	BAAI/bge-base-en-v1.5 — local, no API key
Reranker	BAAI/bge-reranker-base cross-encoder — local
Vector store	ChromaDB (persistent, on disk)
Graph store	NetworkX serialised to JSON under <code>.graphs/</code>
Relational store	SQLite (backend/data/b2b.sqlite3) for the B2B layer
API surface	25 routes — 15 core, 10 B2B
Required secrets	GROQ_API_KEY, GITHUB_TOKEN

1 · Executive summary

What the product does. A developer points the system at a GitHub repository. The system reads every indexable file once and builds two independent indexes from it. From then on it can answer natural-language questions about the codebase with citations, recommend an issue matched to a developer's stated skills, generate a graph-informed reading order, review a diff for readiness, and — the feature the design documents treat as the flagship — compute the *blast radius* of a change: which files break if these files change, derived from parsed imports rather than guessed by a model.

The central design thesis. Use the deterministic graph for anything that has a correct answer, and use the language model only to translate the question and explain the result. Blast radius, test selection, and reviewer routing all involve **zero** model calls. This is what separates the product from a wrapper around a chat completion.

The B2B layer. A second surface reuses that identical engine with no changes to parsing, graph, or retrieval logic, repositioned for engineering leadership: test selection to cut CI wait time, reviewer routing by real code ownership, a team roster with per-member progress, and a governance report packaging evaluation evidence for a compliance reviewer.

Verified working end-to-end

The following were exercised against a live server with real credentials, real GitHub API calls, and a real repository (martinmimigames/tiny-music-player, 19 of 49 files indexed, 45 chunks):

- **POST /index** — full pipeline: GitHub fetch, tree-sitter parse, graph build, local embedding, ChromaDB write. Returned 200 OK.
- **POST /ask** — council gate classified the query and routed it (route: vector), retrieval and reranking ran, and a correct, well-grounded answer streamed back with five real source files cited.
- **POST /b2b/members/{id}/assign** — proved two external integrations at once: GitHub issue listing and a Groq completion, returning a real matched issue with rationale, then persisting the assignment to SQLite.
- The local embedding model (438 MB) and the reranker model both downloaded and ran on first use with no API key involved.

Verified by type-check and mocked integration only

- All ten B2B endpoints — full organisation, member, repository, tagging, assignment, PR-history, and roster lifecycle — exercised through FastAPI's TestClient with external services stubbed.
- The B2B frontend panels — compile clean under `tsc --noEmit` in strict mode with zero errors and zero new lint findings, but have not been clicked through in a browser.
- **Test selection** and **reviewer routing** were confirmed to return correct errors when a repository is not indexed, but their success paths have not been run against a live indexed repository.

Known gaps

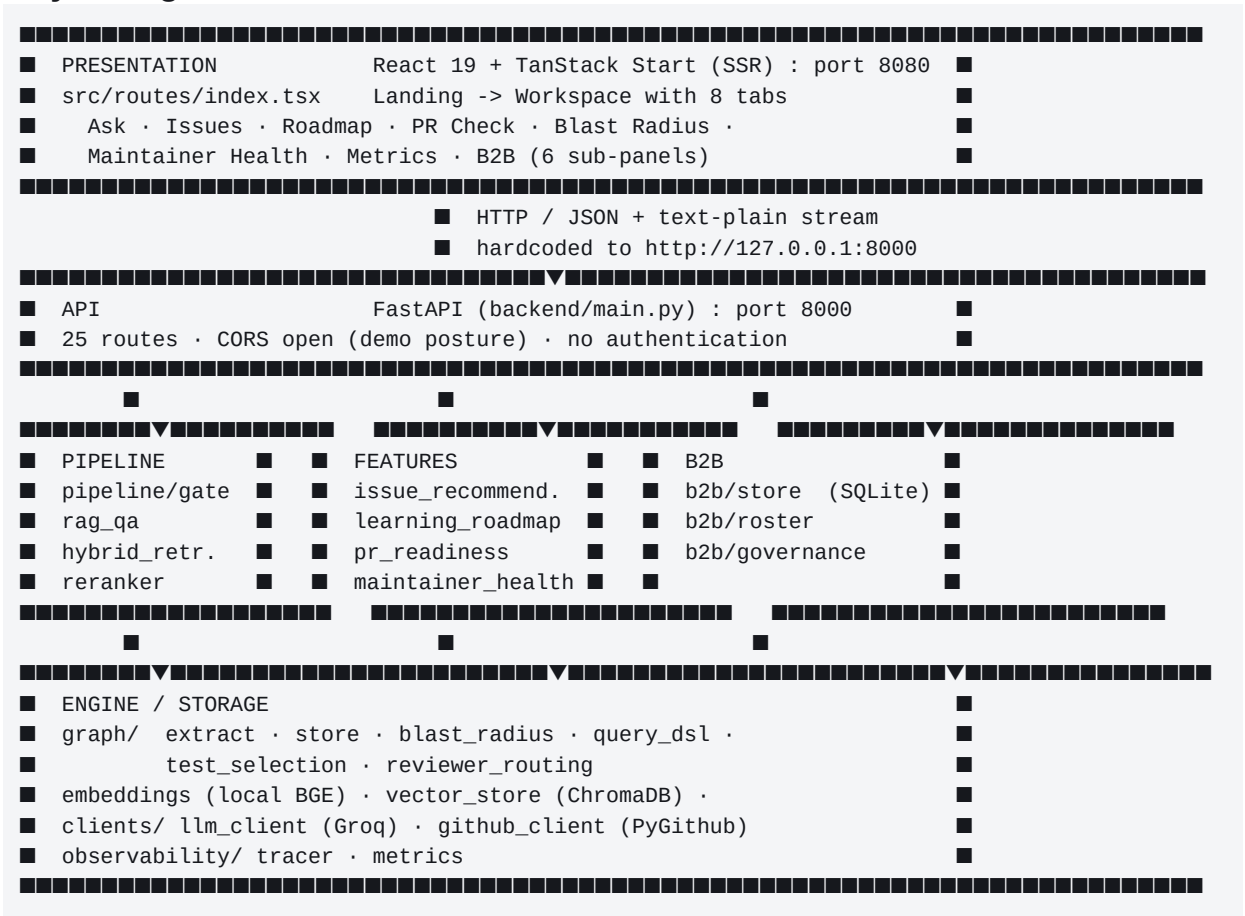
- **The Mutagent optimization loop does not exist in this repository.** Only its output artefacts (prompt files) and its evidence channel (trace logging) are present. See section 8.
- Because no evaluation reports exist, `/metrics` and `/b2b/governance-report` currently render every component with null scores and `has_report: false`.
- There is no authentication anywhere. Organisation and member identifiers are passed as plain request parameters.

- Trace logs retain raw private repository content indefinitely with no rotation or redaction (section 10).

2 · System architecture

The system is four layers. Data flows down on indexing and up on querying.

Layer diagram



The two indexes

A single POST /index call produces two independent representations of the same repository. Everything downstream is a combination of them.

	Dependency graph	Vector index
Built by	tree-sitter parse to NetworkX	chunk to BGE embed to ChromaDB
Stored at	.graphs/<owner>__<repo>.json	ChromaDB dir, collection per repo
Node / record	one node per file	one record per ~500-token chunk
Edges / metadata	A imports B (directed)	path, type, chunk_index
Answers	structural questions with a provably correct answer	semantic questions of meaning and similarity
Model calls	none, ever	none at index time (local model)
Powers	blast radius, test selection, reviewer routing, roadmap centrality	RAG answers, HyDE retrieval

Why two stores rather than one: a question like “*what breaks if I change auth.py?*” has a correct answer that can be computed by graph traversal. Asking a language model to guess it would be strictly worse —

slower, more expensive, and wrong in ways nobody can audit. Conversely “*how does authentication work here?*” has no single correct file set and needs semantic retrieval. The system routes each class of question to the representation that can actually answer it.

3 · Repository layout

```

MUTAV2/
■■■■ src/routes/index.tsx      entire frontend – 8 tabs, ~2200 lines
■■■■ backend/
■   ■■■■ main.py              FastAPI app · all 25 routes
■   ■■■■ config.py           Settings singleton · load_prompt() · repo_id_from_url()
■   ■
■   ■■■■ indexing.py         /index orchestrator: graph build + vector build
■   ■■■■ github_client.py    module-level GitHub helpers (fetch_repo_files, rate limit)
■   ■■■■ embeddings.py       chunk_text() + embed_texts() [local BGE]
■   ■■■■ vector_store.py     ChromaDB wrapper · collection per repo (md5 of URL)
■   ■
■   ■■■■ rag_qa.py            HyDE · fan-out · context build · streaming synthesis
■   ■■■■ hybrid_retriever.py dense + graph-sparse + RRF fusion + graph-fact injection
■   ■■■■ reranker.py         local bge-reranker-base cross-encoder
■   ■■■■ pipeline/gate.py    AI-Council gate – classification + routing + query splitting
■   ■
■   ■■■■ issue_recommendation.py issue matching (prompt: issue_rec)
■   ■■■■ learning_roadmap.py  reading order (prompt: roadmap)
■   ■■■■ pr_readiness.py     diff review (prompt: pr_check)
■   ■■■■ maintainer_health.py issue triage (prompt: issue_health)
■   ■
■   ■■■■ graph/
■   ■   ■■■■ extract.py      tree-sitter: identifiers, imports, definitions, tiers
■   ■   ■■■■ store.py        build_graph · save_graph · load_graph (LRU) · delete_graph
■   ■   ■■■■ blast_radius.py reverse BFS + identifier-occurrence union
■   ■   ■■■■ query_dsl.py    6-operation query contract
■   ■   ■■■■ test_selection.py blast radius filtered to test files
■   ■   ■■■■ reviewer_routing.py blast radius x git authorship
■   ■
■   ■■■■ b2b/
■   ■   ■■■■ store.py        SQLite – 7 tables
■   ■   ■■■■ roster.py       manager view composition
■   ■   ■■■■ governance.py   /metrics reshaped as compliance evidence
■   ■
■   ■■■■ clients/
■   ■   ■■■■ llm_client.py   Groq completion · retry/backoff · JSON extraction
■   ■   ■■■■ github_client.py class-based PyGithub wrapper
■   ■
■   ■■■■ observability/
■   ■   ■■■■ tracer.py       append-only JSONL evidence log
■   ■   ■■■■ metrics.py      reads mutagent/reports/*.delta.json
■   ■
■   ■■■■ mutagent/
■   ■   ■■■■ prompts/        6 files – THE SINGLE SOURCE OF TRUTH for all prompts
■   ■   ■■■■ traces/         append-only JSONL per component
■   ■
■■■■ .graphs/                persisted dependency graphs (JSON)
■■■■ CHANGES.md             full changelog of work on this branch
■■■■ B2B_AUDIT.md            data-handling audit (plan section 6)

```

Two structural conventions matter

- **Prompts are never inlined in Python.** Every prompt lives in backend/mutagent/prompts/*.txt and is read at call time via config.load_prompt(name). The stated reason is that an earlier design kept prompts in two places that had to be manually synchronised and one was always stale. A practical consequence: editing a prompt file changes production behaviour on the next request with no code change and no restart.

- **repo_id_from_url()** is the one canonical identifier derivation.
https://github.com/owner/repo becomes owner__repo, used for graph filenames and by the vector store. Its docstring warns that changing it invalidates every existing index and graph.

4 · Workflow: indexing a repository

```
POST /index · body {"repo_url": "..."} · entry point indexing.index_repo()
```

This is the only write path. Every other feature reads what this produces.

Step 1 — Fetch

`github_client.fetch_repo_files()` walks the default branch breadth-first. A file is kept only if its extension is in a fixed allow-list of 22 text-like extensions (source, markdown, JSON, YAML, CSS, HTML) *and* it is under 200 KB — the size cap exists to skip generated bundles and lockfiles. The function returns the kept files plus a total count, which is why the UI can report “19 of 49 files indexed”.

Step 2 — Build the dependency graph

For each file, `graph/extract.extract_file()` parses the source with tree-sitter and produces three things: the set of **identifiers**, the list of **import paths**, and the list of **definitions** (name, kind, line). Each file is assigned a **coverage tier**, which is the system's mechanism for being honest about how much it actually knows about a file:

Tier	Meaning	Languages
deep	Hand-written import <i>and</i> definition queries exist. Real import edges are extracted.	15: python, javascript, typescript, tsx, java, go, rust, c, cpp, c_sharp, ruby, php, kotlin, html, css
occurrence-only	The grammar loads and identifiers are extracted, but no import query exists — so no edges.	e.g. swift, scala, lua, elixir
unparseable	No grammar available. The file is a node with no attributes.	remainder

`graph/store.build_graph()` then adds one node per file and resolves each import string to a real repository path using a four-strategy cascade: exact path match, dot-notation to path with language-appropriate extensions (plus a package `__init__` attempt), suffix match, and bare filename match. Unresolved imports are dropped silently and correctly — an import of `requests` or `react` is an external dependency, not an internal edge. Only deep-tier files contribute edges. The graph is serialised to `.graphs/owner__repo.json`.

Failure posture at this step

The entire graph build sits inside a `try/except` that prints a warning and continues. If parsing fails the repository is still fully usable for semantic Q&A — only the structural features degrade. This is deliberate, but it does mean a graph build failure is easy to miss in the logs.

Step 3 — Build the vector index

`embeddings.chunk_text()` splits each file into ~500-token chunks with 50 tokens of overlap, measured with the `cl100k_base` tokenizer so chunk size is consistent regardless of language or formatting. Chunks are embedded in batches of 100 by `embed_texts()` using a lazily-loaded local BAAI/bge-base-en-v1.5 sentence-transformers model with L2 normalisation, and written to a ChromaDB collection named `repo_<md5(repo_url)>` — hashed because Chroma requires collection names between 3 and 63 characters. Each record carries `{path, type, chunk_index}` as metadata; the `path` field is what later enables graph-filtered vector search.

Embeddings are local — a deliberate correction

This step previously called Gemini's hosted embeddings API, which contradicted the project's own self-hosted deployment claim and required a third API key. It now runs a local model: no key, no quota, no rate limit, and indexing costs zero external API calls. The model downloads once (~438 MB) and is cached.

Step 4 — Return

```
{"files_indexed": 19, "chunks_created": 45, "total_repo_files": 49}
```

A matching DELETE `/index?repo_url=...` purges both representations — the Chroma collection and the graph JSON. It was added because before it there was no way to delete indexed data at all. Nothing calls it automatically; there is no session or organisation boundary that triggers a wipe.

A short user question and the code that answers it often sit far apart in embedding space — the question is terse and interrogative, the answer is dense and declarative. HyDE closes that gap: the model first drafts a short passage that *would* answer the question, written in the style of source comments or documentation, and **that passage is embedded instead of the question**. The draft is never shown to anyone; it exists purely to land closer to the real chunks in vector space.

- Prompt: `prompts/hyde.txt` · temperature 0.3 · max 300 tokens.
- Cached on the question text, same reasoning as the gate.
- **Non-fatal**. Wrapped so that any failure falls back to embedding the raw question — a retrieval-quality boost, not a hard dependency.
- Toggled by `HYDE_ENABLED` (default on).
- The **original** question, not the HyDE passage, still drives symbol extraction, reranking, and the final answer.

Stage 3 — Hybrid retrieval

`hybrid_retriever.hybrid_retrieve()` combines three signals and makes no model calls at all.

(a) Dense arm. Standard vector similarity search over the Chroma collection using the HyDE embedding, returning `TOP_K` chunks.

(b) Graph-aware sparse arm. Identifiers are extracted from the question with a regex, filtered against a stop-list of ~30 common words so that “*how does login work*” does not treat `login` as a symbol. Surviving symbols are looked up in the graph via the `definition_of` and `occurrences_of` query operations, producing a set of file paths. A **second** vector search then runs restricted to those paths. This is the mechanism that ties the two indexes together: the graph narrows *where* to look, and vector similarity ranks *what* to return within that narrowed set.

(c) Reciprocal Rank Fusion. The two result lists are merged with RRF ($1/(k+rank)$, $k=60$), keyed on `(path, start_line, end_line)`. RRF is used because the two arms produce incomparable scores — cosine distance versus a filtered cosine distance — and rank-based fusion sidesteps the need to normalise them against each other.

(d) Deterministic graph-fact injection. If the question contains *break*, *impact*, or *rely*, the retriever computes a real blast radius and prepends it as a synthetic top-ranked chunk of plain text. The model is then explaining a fact rather than inventing one — the design thesis applied inside the retrieval layer.

The gate's route decision is honoured here: `route="vector"` skips (b) and (d) entirely. `graph` and `hybrid` both run the full pipeline — `graph` is treated as `hybrid` rather than being under-served by a `graph-only` path that does not exist.

Stage 4 — Reranking

A local `BAAI/bge-reranker-base` cross-encoder scores every (question, chunk) pair jointly and keeps the top `RERANK_TOP_K` (default 5) out of `TOP_K` (default 25). A cross-encoder is far more accurate than embedding similarity because it sees both texts together, but it is too slow to run over a whole corpus — hence the retrieve-wide-then-rerank-narrow shape. Runs locally; no API call.

Stage 5 — Multi-intent fan-out

When the gate splits a compound question — “*how does auth work **and** where is rate limiting configured?*” — into several distinct sub-questions, `_build_context()` runs stages 2 through 4 **separately for each sub-question**, each with its own route, then de-duplicates the merged chunks on `(path, start_line, end_line)`. A single retrieval pass over a compound question tends to surface chunks only for whichever intent dominates the embedding; fanning out fixes that. A single-intent question takes exactly the previous

one-pass path with no behavioural change.

Stage 6 — Streamed synthesis and the response contract

The final call streams. The endpoint therefore has two response shapes, distinguished by Content-Type:

Outcome	Content-Type	Body
Gate rejects	application/json	{answer, sources: [], passed: false, reason, classification, route, sub_queries}
Gate passes	text/plain (streamed)	answer text chunks, then a \n\n<<META>>>{json} trailer carrying sources, route, sub_queries

Retrieval and the first token are pulled *before* the streaming response opens, so an unindexed repository still surfaces as a clean 400 rather than a broken stream. The frontend reads the stream incrementally, growing one message bubble as text arrives and splitting the trailer off at the end.

6 · The deterministic graph features

Zero model calls in this entire section. These are the answers the product can prove.

6.1 Blast radius — POST /blast-radius

What breaks if these files change? The answer is the union of two computations:

- **Reverse breadth-first search over import edges.** The graph is reversed and traversed outward from the changed files, so each hop answers “who imports this?”. Hop count is recorded per file and capped by `max_hops` (default 3, hard cap 10).
- **Identifier-occurrence union.** Any file sharing an identifier with a changed file is included at `hops=0`. This catches real coupling that import parsing misses, at the cost of false positives.

Results are labelled `kind: "import" or "occurrence"` and sorted by `(kind_rank, hops, path)`, so parsed edges — the trustworthy signal — always rank above name matches. The response carries a `coverage_tier` equal to the **weakest** tier among the targets, which tells the caller how much to trust the result. Errors are returned as values, never raised: an ambiguous or missing target produces a result object with `error` set. Partial paths resolve when unambiguous — `conf.py` finds `docs/conf.py`, but two matches produce an explicit ambiguity error rather than a guess.

6.2 Query DSL — the contract layer

`graph/query_dsl.execute_query(query, graph)` exposes six operations behind one deterministic interface: `blast_radius`, `importers_of`, `imports_of`, `definition_of`, `occurrences_of`, `neighbors`. Its contract: never raise for a bad query, identical inputs always yield identical output, unknown fields ignored, unknown operations return an error value, hard caps of 10 hops and 1000 results. This is the interface the hybrid retriever calls for its sparse arm.

6.3 Test selection — POST /b2b/test-selection

Blast radius intersected with a test-file pattern set: 14 filename regexes across Python, Go, JavaScript/TypeScript, Java, Ruby, Kotlin, Rust, and C#, plus seven directory segments (`tests`, `__tests__`, `spec`, ...). Returns the selected tests, the repository total, and a ratio string such as “4 of 200 tests”.

The product framing is deliberately limited

The response always carries a disclaimer: run these first, then the full suite on merge. Reflection, dynamic dispatch, and runtime wiring create edges a static parse cannot see. This is test *selection*, not test *proving* — and the endpoint says so in its own payload rather than leaving the caller to infer it.

6.4 Reviewer routing — POST /b2b/reviewer-routing

Computes the blast radius, then queries GitHub commit history for each affected file (capped at 50 files and 10 commits per file to bound API cost). Authors are scored by inverse recency — `1 / days_ago` — so someone who touched the code last week outranks someone who touched it in 2019. The PR author is excluded from their own reviewer list. The distinction from a CODEOWNERS file is the point: this routes to whoever owns the code in the **impacted set**, not merely whoever is tagged on the changed file itself.

6.5 Graph-informed roadmap — POST /roadmap

Files are ranked by **in-degree** — how many other files import them — and the top eight are passed to the model as “structurally central”. The graph decides what central means; the model only writes prose around

that ranking. When no graph exists the prompt is explicitly told to say so rather than guess at internals it cannot see: the system degrades loudly instead of silently.

7 · The model-backed features

All four follow one identical pattern, which is worth stating once: `load_prompt(name)` to read the prompt from disk, `.format(...)` to substitute, `complete()` at temperature 0.1 through Groq, `trace()` to append evidence, `extract_json()` to parse, then validate the result against reality before returning it.

7.1 Issue recommendation — `/recommend-issue`

Fetches up to 30 open issues and formats them as `Issue #N: 'title' [labels]` — deliberately no body text. The model returns an issue id and rationale.

Two designed behaviours worth noting

Hallucination grounding: the returned `issue_id` is checked against the issues actually shown to the model. An id that was never in the list is discarded and treated as no match, rather than passed through to the user.

Safe refusal: `issue_id: null` is a successful outcome meaning no issue is an appropriate fit — not an error. Recommending a badly-matched issue to a newcomer is worse than recommending none.

7.2 PR readiness — `POST /pr-check`

Deterministic facts are computed **first**, in Python, by regex over the diff: `has_tests`, `touches_docs`, `diff_size_lines`, `is_large_diff` (over 300 lines), and `touches_security_sensitive_path` (matching `auth`, `permission`, `session`, `crypto`, `secret`, `credential`, `admin`, `password`, `token`). Those facts are then handed to the model alongside the diff, so the model is reasoning over measured evidence rather than counting lines itself. The verdict is constrained to `ready`, `needs_changes`, or `blocked`, and anything else is rejected as unparseable.

7.3 Maintainer / team health — `/maintainer-health`, `/b2b/team-health`

Classifies each open issue as `healthy`, `mislabelled`, `vague`, or `stale`, given the issue text plus computed signals (`days_open`, `comment_count`, `has_linked_pr`). The **`mislabelled`** verdict is the differentiating one: an issue tagged *good first issue* whose actual content requires deep domain knowledge. Returns a health score (fraction healthy), the `mislabelled` list with rationales, and median response latency. Verdicts outside the four valid strings are discarded rather than counted.

`/b2b/team-health` is the identical function behind a second route — zero new logic. The separate route exists so the internal-process-health framing is legible in the API surface itself.

7.4 The Groq client

`clients/llm_client.py` centralises every non-RAG model call. It retries up to five times on transient errors (rate limit, connection, timeout, internal server error) with exponential backoff, and honours a server-provided `retry-after` header when present, capped at 60 seconds. Its `extract_json()` is deliberately forgiving — it strips fenced code blocks, tolerates leading prose and trailing commentary, and falls back to the outermost brace pair — because small models wrap JSON in explanation regularly. It returns `None` rather than raising, leaving each caller to decide what a parse failure means for it.

8 · Mutagent: design intent versus what exists here

This section deserves the most care, because the project's design documents describe Mutagent as “the self-improvement engine for the entire system”, and the question “*how is Mutagent working in each step?*” has a precise and somewhat uncomfortable answer.

8.1 What Mutagent is designed to be

Mutagent is described as a **measurement system, not a decision-maker**. It does not know what “good” means; the developer defines it through a rubric, supplies a dataset of test cases, and names a prompt to improve. Mutagent then runs a genetic optimization loop:

```
baseline prompt
-> evaluate against every case in the dataset
-> score each case with the rubric
-> ask the model to mutate the prompt into N variants
-> re-evaluate and score every variant
-> keep the highest scorer
-> repeat for G generations
-> write reports/<target>.delta.json (baseline vs optimized)
-> overwrite prompts/<target>.txt with the winner
```

Critically, this loop is designed to run **offline during development, once per target** — never on a user request. In production the optimized prompt is simply a static file read from disk. The design documents are emphatic on this point: when a user asks a question, Mutagent does not run.

8.2 The six intended evaluation targets

#	Target	Production home	Eval signal	Stated baseline
1	issue_rec	issue_recommendation.py	LLM judge, 3 criteria	15/22 (68%)
2	council	pipeline/gate.py	LLM judge, 4 criteria	18/24 (75%)
3	graph_query	graph/query_dsl.py	executable — set F1 vs reference	machine-generated
4	router	pipeline/gate.py (route only)	LLM judge, routing only	23 cases
5	pr_check	pr_readiness.py	deterministic policy match	23 cases
6	issue_health	maintainer_health.py	LLM judge, 3 criteria	16/22 (73%)

Targets 2 and 4 deliberately share one prompt but use different datasets and rubrics, so that two distinct failure modes stay separable: the gate wrongly rejecting good queries, versus the router choosing the wrong retrieval strategy. Collapsing them into one target would conflate the two.

8.3 What actually exists in this repository

The optimization loop is not present in this codebase.

A full inventory of backend/mutagent/ finds **six prompt files and one trace file**. There is no harness.py, no run.py, and no datasets/, rubrics/, or reports/ directory. The command the documentation centres on — `python -m mutagent.run <target> --optimize` — cannot run here, because the module does not exist.

Component	Designed	Present here
prompts/*.txt	6 targets + roadmap	Yes — 6 files: council, hyde, issue_health, issue_rec, pr_check, roadmap
traces/*.jsonl	one per component	Partly — the writer works and is wired into every feature, but only <code>issue_health.jsonl</code> exists, with a single line
datasets/*.json	6 datasets, ~130 cases	No — directory absent
rubrics/*.json	6 rubrics	No — directory absent
reports/*.delta.json	baseline vs optimized	No — directory absent
harness.py	evaluate() + optimize()	No
run.py	the CLI	No

8.4 So how does Mutagent participate in each step, concretely?

Two of the three mechanisms are real and load-bearing; the third is absent.

Mechanism 1 — Prompt sourcing (real, and used on every request)

Every model-backed step reads its prompt from `backend/mutagent/prompts/` at call time. This is the sense in which Mutagent is “in” each step: the prompt files *are* the artefact a Mutagent run would produce, and production consumes them directly with no duplication in code.

Step in a request	Prompt file	Loaded by
Council gate / router	council.txt	pipeline/gate.py
HyDE draft	hyde.txt	rag_qa.py
Issue recommendation	issue_rec.txt	issue_recommendation.py
PR readiness	pr_check.txt	pr_readiness.py
Issue / team health	issue_health.txt	maintainer_health.py
Learning roadmap	roadmap.txt	learning_roadmap.py

A practical consequence: because prompts are read from disk per call, dropping an optimized prompt file into that directory changes production behaviour immediately, with no code change. That is exactly the hand-off the architecture was designed for — the receiving half is built even though the producing half is not.

Mechanism 2 — Trace emission (real, and wired everywhere)

`observability/tracer.trace()` appends one JSON line per model call to `mutagent/traces/<component>.jsonl`, recording timestamp, component, the fully instantiated prompt, the raw unparsed output, latency in milliseconds, and per-component extras. Raw output is stored exactly as received — malformed JSON survives as evidence rather than being swallowed by defensive parsing. Traces are emitted by the council gate, issue recommendation, PR check, issue health, and roadmap. This is Mutagent’s **evidence channel**: it feeds an audit, not an optimizer, and nothing reads it back into the running system.

One gap: the RAG path (`rag_qa.py` — HyDE and answer synthesis) calls `trace()` nowhere. It is simultaneously the path that injects the most real source code into a prompt and the only one with no audit trail.

Mechanism 3 — Optimization and scoring (absent)

No prompt in this repository has been measured, scored, or optimized by any code present in this repository. The baseline figures quoted in the design documents — 15/22, 18/24, 16/22 — are not reproducible here, because neither the datasets that would produce them nor the harness that would run them exists. The prompt files are hand-written, not machine-selected.

8.5 The observable consequence

`observability/metrics.get_metrics()` iterates a hard-coded list of the six targets and, for each, attempts to read `reports/<id>.delta.json`. Since that directory does not exist, every target returns `has_report: false` with null baseline, optimized, and delta values. Therefore:

- The **Metrics** tab renders all six rows with em-dashes in every score column.
- `/b2b/governance-report` — the artefact intended for a compliance reviewer — reports every component as `evaluated: false`.
- Trace counts are the only non-null numbers, and only for components that have actually run.

To the system's credit, the governance endpoint does not paper over this: it emits `dataset_version` and `rubric_version` as explicit nulls with a written caveat, rather than fabricating version strings, and its `generation_model_note` reports the model actually configured rather than asserting the design's single-model constraint unconditionally.

What it would take to close this gap

The receiving architecture is complete — prompt loading, trace emission, report reading, and a metrics surface all exist and work. What is missing is the producing half: per-target datasets and rubrics, an evaluate/optimize harness, and a CLI. Until those exist, the accurate description of this system is “a well-instrumented application whose prompts are centralised and traceable”, not “a self-improving system”. Both are defensible; only one is true today.

9 · The B2B / enterprise layer

Identical engine, different buyer and unit of value. No parsing, graph, or retrieval logic changes.

9.1 Positioning

Open-source framing	Enterprise framing
Onboard to an unfamiliar repository	New-hire ramp-time reduction
What should I work on?	Ticket routing by demonstrated skill
What will I break?	Change impact to test selection and reviewer routing (the budget line)
Reliability dashboard	AI governance evidence
Buyer: individual contributor	Buyer: engineering leadership, managers, security and compliance

9.2 Data model

Plain `sqlite3` — no ORM, consistent with the codebase's preference for the simplest store that works (Chroma for vectors, JSON for the graph). Seven tables in `backend/data/b2b.sqlite3`, created on import:

Table	Purpose
<code>organizations</code>	name plus policy settings (test-selection threshold, reviewer routing toggle)
<code>members</code>	name, email, skill profile; unique per (org, email)
<code>org_repos</code>	which repositories an organisation has registered
<code>tagged_issues</code>	team-lead curation: tag plus subsystem, upserted on conflict
<code>assigned_issues</code>	recommendation results persisted against a member
<code>pr_readiness_history</code>	time series of verdicts per member
<code>member_roadmap_status</code>	<code>not_started</code> / <code>in_progress</code> / <code>completed</code> per (member, repo)

The last table is an addition beyond the written plan. The plan calls for “per-member roadmap progress”, but this application's roadmap is stateless free-text prose, not a checklist — so progress is tracked as a coarse status rather than inventing a percentage the rest of the system cannot substantiate.

9.3 Feature composition

The B2B endpoints are mostly **composition**, not new logic — which is the entire architectural claim of the layer:

- `POST /b2b/members/{id}/assign` calls the **unchanged** `recommend_issue()` — including its safe-refusal behaviour — then persists the result against the member.
- `POST /b2b/members/{id}/pr-check` calls the **unchanged** `check_pr_readiness()`, then logs the verdict to build a trend.
- `GET /b2b/orgs/{id}/roster` composes member profile, assignments, roadmap status, and PR-readiness history into one payload, computing open-assignment count and PR-ready rate — replacing three separate queries and manual status-chasing.

- GET `/b2b/governance-report` reshapes the existing `/metrics` payload with last-evaluated timestamps taken from report file `mtimes`. No new evaluation logic whatsoever.

9.4 Frontend

A **B2B** tab was added alongside the seven original tabs, with six sub-panels: Team Setup, Roster, Tag Issues, Test Selection, Reviewer Routing, and Governance. It follows the existing file's conventions exactly — the same `apiFetch` helper and the same `SectionHeader`, `ErrorBox`, `Spinner`, and `Skeleton` primitives, styled with raw Tailwind rather than the `shadcn/ui` component library that sits unused in `src/components/ui/` (the original panels do not use it either). No panel was built for `/b2b/team-health`, since it is a zero-logic alias whose response shape the Maintainer Health tab already demonstrates.

10 · Configuration, deployment, and data handling

10.1 Configuration

Variable	Default	Effect
GROQ_API_KEY	—	Required. Every LLM call except RAG-with-Ollama
GITHUB_TOKEN	—	Strongly recommended; public repos work without it at low rate limits
EMBEDDING_MODEL	BAAI/bge-base-en-v1.5	Local sentence-transformers model; no key
LLM_MODEL	llama-3.1-8b-instant	Groq generation model
OLLAMA_BASE_URL	(blank)	If set, routes <i>only</i> HyDE and RAG synthesis to Ollama. Must include the /v1 suffix
OLLAMA_MODEL	qwen2.5:3b	Model name on that Ollama instance
TOP_K / RERANK_TOP_K	25 / 5	Retrieve-wide, rerank-narrow
CHUNK_SIZE / OVERLAP	500 / 50	Token-measured chunking
HYDE_ENABLED	true	Set false to embed the raw question instead

10.2 Two environment pitfalls found by running it

- `httpx<0.28` **must be pinned**. `openai==1.51.0`'s internal client passes a `proxies` keyword that `httpx` removed in 0.28. Unpinned, a fresh install breaks every LLM call with a `TypeError` at client construction. Now pinned in `requirements.txt`.
- **ngrok tunnels need a skip header**. ngrok's free tier intercepts unheadered requests with an HTML interstitial. The OpenAI SDK sends no custom headers by default, so every completion through a tunnelled Ollama silently received HTML where JSON was expected. Fixed by passing `default_headers={"ngrok-skip-browser-warning": "true"}`.

10.3 Network boundary

Destination	Called from	When
api.github.com	github_client (both)	every repo, issue, and commit lookup
api.groq.com	clients/llm_client, rag_qa	every LLM call (unless Ollama configured for RAG)
Ollama (optional)	rag_qa only	only if OLLAMA_BASE_URL is set
huggingface.co	embeddings, reranker	first use only , then fully local
openaipublic.blob...	tiktoken	first use only , then fully local

A genuinely air-gapped deployment needs the embedding model, the reranker, and the tokenizer data pre-cached into the image — “self-hosted” is not sufficient on its own.

10.4 Data-handling audit findings

#	Finding	Status
1	Trace logs are append-only, never rotated, not scoped by organisation. PR check writes up to 4,000 characters of raw diff; issue health writes full issue bodies.	Open — needs a redaction and retention policy decision
2	No deletion path existed for indexed data at all.	Fixed — DELETE /index now purges both stores; verified the graph file is removed from disk. No automatic trigger yet
3	One global GitHub token shared by every organisation, though the data model implies per-organisation credentials.	Open by choice — a plaintext per-org token column would be security theatre; needs a real secrets manager
4	The plan's self-hosted claim assumed local embeddings; the code called a hosted API.	Fixed — embeddings are now local, and the claim matches the code

Findings 1 and 3 still block presenting the governance report as compliance-ready without the stated caveats. Neither blocks a demo.

11 · API reference and verification status

Core routes

Route	Purpose	Status
POST /index	Build graph + vector index	Live-verified
DELETE /index	Purge both stores	Verified (graph deletion confirmed on disk)
POST /ask	Council-gated, streamed hybrid RAG	Live-verified
POST / GET /recommend-issue	Issue matching	Live-verified (via B2B assign)
POST /roadmap	Graph-informed reading order	Not run live
POST /pr-check	Diff readiness	Mocked only
POST /blast-radius	Change impact	Not run live
GET /maintainer-health	Issue triage	Not run live (one trace exists)
GET /metrics	Evaluation scores	Live — returns nulls, no reports exist
GET /health, /rate-limit, /	Health and quota	Live-verified

B2B routes

Route	Purpose	Status
POST /b2b/test-selection	Impacted tests	Error path verified; success path not run live
POST /b2b/reviewer-routing	Reviewers by ownership	Error path verified; success path not run live
POST / GET /b2b/orgs	Organisations	Mocked lifecycle verified
POST / GET .../members	Members	Mocked lifecycle verified
POST / GET .../repos	Repository registration	Mocked lifecycle verified
POST / GET .../issues/tag(s)	Team-lead tagging	Mocked lifecycle verified
POST /b2b/members/{id}/assign	Ticket routing	Live-verified — GitHub + Groq
POST /b2b/members/{id}/pr-check	Logged PR check	Mocked only
POST .../roadmap-status	Progress tracking	Mocked; validation path verified
GET /b2b/orgs/{id}/roster	Manager view	Mocked; aggregation verified
GET /b2b/team-health	Team health alias	Mocked only
GET /b2b/governance-report	Compliance evidence	Verified — renders with nulls

How each status was established

- **Live-verified** — executed against a running server with real Groq and GitHub credentials and a real repository, returning correct data.

- **Mocked** — executed through FastAPI's TestClient with external services stubbed. Proves wiring, validation, persistence, and response shape; does not prove third-party integration.
- **Not run live** — the code path exists and imports cleanly but has not been exercised end-to-end.

Frontend verification: `tsc --noEmit` passes with zero errors in strict mode across the whole route file, and ESLint reports zero new findings. The B2B panels have **not** been clicked through in a browser; the original seven tabs were confirmed rendering.

12 · Summary judgement

What is genuinely built and working: a two-representation indexing pipeline with local embeddings and no API-key dependency for retrieval; a six-stage gated RAG pipeline with HyDE, hybrid graph-aware retrieval, cross-encoder reranking, multi-intent fan-out, and streaming; a deterministic dependency-graph engine backing blast radius, test selection, reviewer routing, and roadmap centrality with zero model calls; four model-backed features with grounded output validation and safe-refusal behaviour; and a complete B2B layer that is composition over that engine rather than a parallel implementation.

What is not built, stated plainly: the Mutagent optimization loop. Its *interfaces* are complete and load-bearing — prompts are centralised on disk and read per call, traces are emitted, reports are read and surfaced — but the harness, datasets, rubrics, and reports that would make the system self-improving are absent, and every score the dashboards display is consequently null. The honest description of this system today is a well-instrumented, well-separated code-intelligence application that is *ready* to be measured, not one that has been.

Recommended next steps, in order

1. Build the missing datasets/ and rubrics/ plus a minimal evaluate harness — target 3 (graph_query) first, since it has executable ground truth and needs no LLM judge, making it the most defensible evidence available.
2. Add `trace( )` calls to the RAG path, closing the one audit blind spot.
3. Live-verify blast radius, test selection, and reviewer routing against an indexed repository — they are the flagship features and are currently the least exercised.
4. Address audit findings 1 and 3 before any external compliance conversation.